

NETWORK INFO

PICKING AN IMAGE

DEMO





NETWORK INFO

Expo Network

- A library that provides access to the device's network such as its IP address, MAC address, and airplane mode status
- Installation
npx expo install **expo-network**
- Methods
 - Network.**getIpAddressAsync()**
 - Network.**getNetworkStateAsync()**
 - Network.**isAirplaneModeEnabledAsync()**

NetworkState

Name	Type	Description
isConnected (optional)	boolean	If there is an active network connection. Note that this does not mean that internet is reachable. This field is <code>false</code> if the type is either <code>Network.NetworkStateType.NONE</code> or <code>Network.NetworkStateType.UNKNOWN</code> , <code>true</code> otherwise.
isInternetReachable (optional)	boolean	If the internet is reachable with the currently active network connection. On Android, this depends on <code>NetInfo.isConnected()</code> (API level < 29) or <code>ConnectivityManager.getActiveNetwork()</code> (API level >= 29). On iOS, this value will always be the same as <code>isConnected</code> .
type (optional)	<code>NetworkStateType</code>	A <code>NetworkStateType</code> enum value that represents the current network connection type.

```
import * as Network from 'expo-network';

const NetworkInfoDemo = () => {
  const [address, setAddress] = useState('');
  const [networkState, setNetworkState] = useState('');
  const [isAirplaneMode, setAirplaneMode] = useState(false);

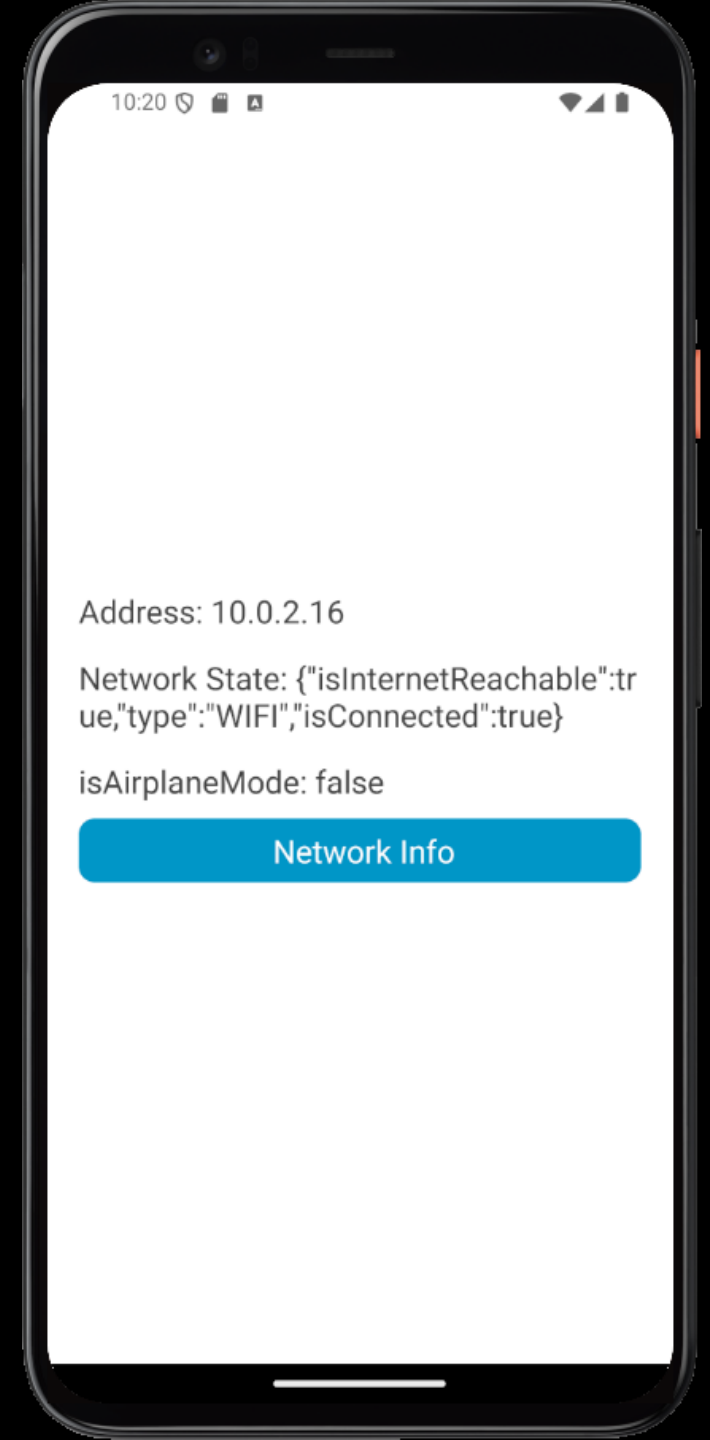
  const getNetworkInfo = async () => {
    let address = await Network.getIpAddressAsync();
    setAddress(address)
    console.log('address', address)

    let state = await Network.getNetworkStateAsync();
    setNetworkState(JSON.stringify(state))

    console.log('networkState', JSON.stringify(state))

    let isAirplaneMode = await Network.isAirplaneModeEnabledAsync();
    setAirplaneMode(isAirplaneMode)
    console.log('isAirplaneMode', isAirplaneMode)

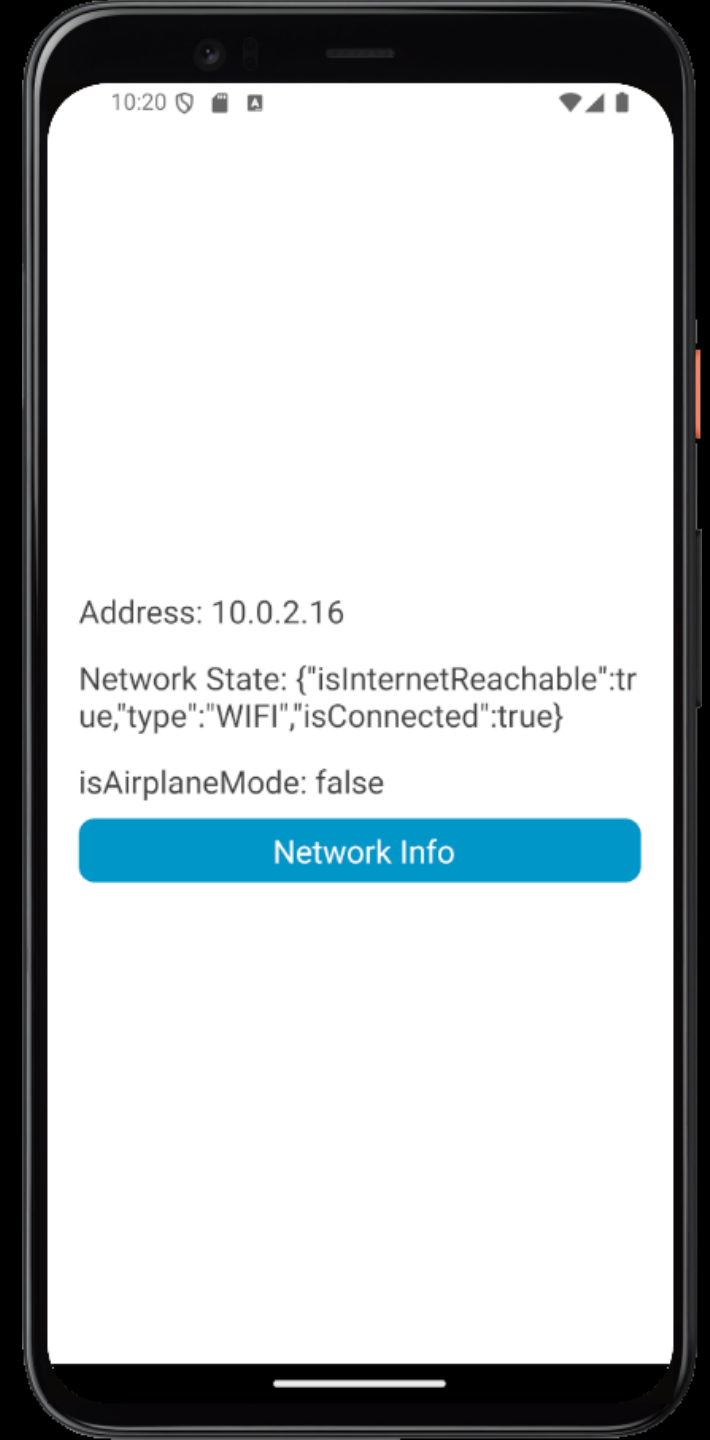
  }
  return (
    )
}
```



```
<View style={styles.container}>
  <Text style={styles.text}>Address: {address}</Text>
  <Text style={styles.text}>Network State:
                                {networkState}</Text>
  <Text style={styles.text}>isAirplaneMode:
                                {String(isAirplaneMode)}</Text>

  <TouchableOpacity style={styles.btn}
                    onPress={() => getNetworkInfo()}>
    <Text style={styles.btnText}> Network Info</Text>
  </TouchableOpacity>

</View>
</View>
```



2

PICKING AN IMAGE

ImagePicker

- A library that provides access to the system's UI for selecting images and videos
 - from the phone's library
 - taking a photo with the camera
- Installation

`npx expo install expo-image-picker`

Usage

➤ Library

```
import * as ImagePicker from 'expo-image-picker'
```

➤ Methods

- ImagePicker.**requestCameraPermissionsAsync**()
- ImagePicker.**requestMediaLibraryPermissionsAsync**(writeOnly)
- ImagePicker.**launchCameraAsync**(options)
- ImagePicker.**launchImageLibraryAsync**(options)

ImagePickerSuccessResult

Representing successful pick result

Name	Type	Description
assets	<code>ImagePickerAsset[]</code>	An array of picked assets.
canceled	<code>false</code>	Boolean flag set to <code>false</code> showing that the request was successful.

ImagePickerCanceledResult

Representing canceled pick result

Name	Type	Description
assets	<code>null</code>	<code>null</code> signifying that the request was canceled.
canceled	<code>true</code>	Boolean flag set to <code>true</code> showing that the request was canceled.

ImagePickerAsset

Represents an asset (image or video) returned by the image picker or camera

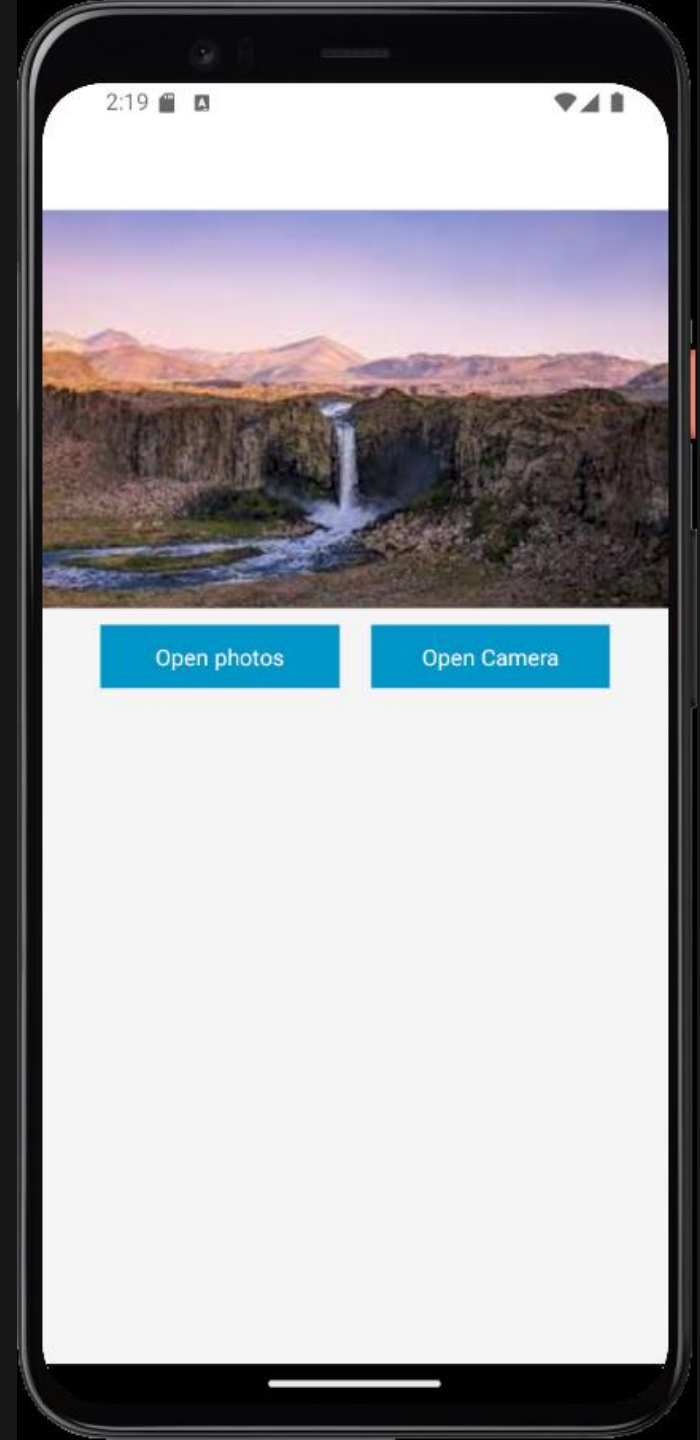
Name	Type	Description
assetId (optional)	string null	Only for:  iOS  Android The unique ID that represents the picked image or video, if picked from the library. It can be used by expo-media-library to manage the picked asset.  This might be <code>null</code> when the ID is unavailable or the user gave limited permission to access the media library. On Android, the ID is unavailable when the user selects a photo by directly browsing file system.
base64 (optional)	string null	When the <code>base64</code> option is truthy, it is a Base64-encoded string of the selected image's JPEG data, otherwise <code>null</code>. If you prepend this with <code>'data:image/jpeg;base64,'</code> to create a data URI, you can use it as the source of an <code>Image</code> element; for example: <pre><Image source={{ uri: 'data:image/jpeg;base64,' + asset.base64 }} style={{ width: 200, height: 200 }} /></pre>
duration (optional)	number null	Length of the video in milliseconds or <code>null</code> if the asset is not a video.
exif (optional)	Record<string, any> null	The <code>exif</code> field is included if the <code>exif</code> option is truthy, and is an object containing the image's EXIF data. The names of this object's properties are EXIF tags and the values are the respective EXIF values for those tags.
fileName (optional)	string null	Preferred filename to use when saving this item. This might be <code>null</code> when the name is unavailable or user gave limited permission to access the media library.
fileSize (optional)	number	File size of the picked image or video, in bytes.
height	number	Height of the image or video.
mimeType (optional)	string	The MIME type of the selected asset or <code>null</code> if could not be determined.
type (optional)	'image' 'video'	The type of the asset.
uri	string	URI to the local image or video file (usable as the source of an <code>Image</code> element, in the case of an image) and <code>width</code> and <code>height</code> specify the dimensions of the media.

```
<View style={styles.container}>

  <Image source={{ uri: selectedImage.localUri }}
    style={styles.logo} />
  <View style={styles.box}>
    <TouchableOpacity onPress={() => openGallery()}
      style={styles.btn}>
      <Text style={styles.btnText}>Open photos</Text>
    </TouchableOpacity>

    <TouchableOpacity onPress={() => openCamera()}
      style={styles.btn}>
      <Text style={styles.btnText}>Open Camera</Text>
    </TouchableOpacity>
  </View>

</View>
```



```

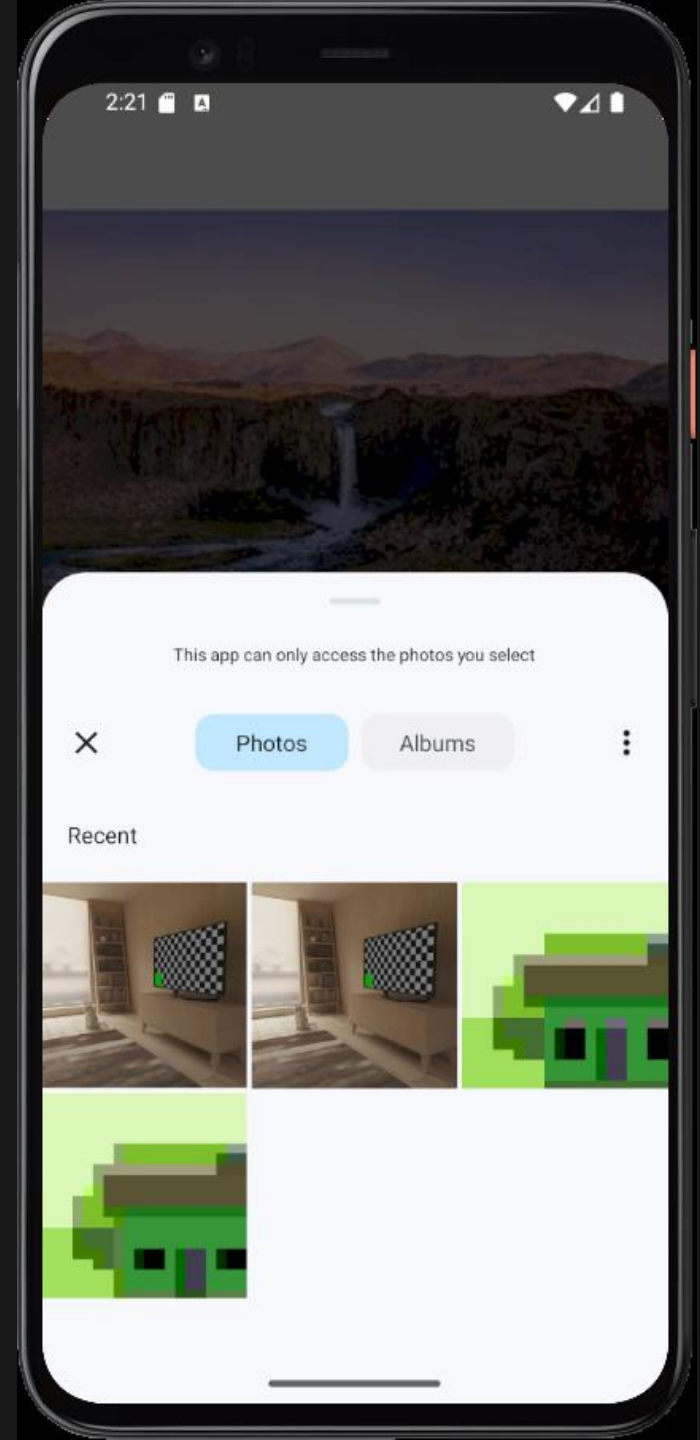
const [selectedImage, setSelectedImage] =
useState({localUri:
"https://firebasestorage.googleapis.com/v0/b/reactnativehosting.appspot.com/o/images%2Fmountain1.jfif?alt=media&token=e7e7f39a-5941-4ea8-86d3-aa08043fb999"
  });

const openGallery = async () =>
{

  let pickerResult = await
  ImagePicker.launchImageLibraryAsync({
    mediaTypes: ImagePicker.MediaTypeOptions.Images,
    allowsEditing: true,
    aspect: [1, 1],
    quality: 1
  });
  if (!pickerResult.canceled)
  {
    console.log(pickerResult.assets[0].uri);
    setSelectedImage({localUri: pickerResult.assets[0].uri });
  }

};

```



```
const [selectedImage, setSelectedImage] =
useState({localUri:
"https://firebasestorage.googleapis.com/v0/b/reactnativehosting.appspot
.com/o/images%2Fmountain1.jfif?alt=media&token=e7e7f39a-5941-4ea8-86d3-
aa08043fb999"
  });

const openCamera = async () => {

  await ImagePicker.requestCameraPermissionsAsync()

  let pickerResult = await ImagePicker.launchCameraAsync({
    cameraType: ImagePicker.CameraType.front,
    allowsEditing: true,
    quality: 1
  });

  if (!pickerResult.canceled)
  {
    console.log(pickerResult.assets[0].uri);
    setSelectedImage({ localUri: pickerResult.assets[0].uri });
  }
}
```



Firestore storage

```
npm install firebase
```


Project Overview


Generative AI


Build with Gemini NEW

Project shortcuts


Storage


Firestore Database


App Hosting NEW

What's new


Data Connect NEW

Product categories

Build ▾

Run ▾

Analytics ▾

Spark
No-cost \$0/month

Upgrade

SDK setup and configuration

☒ npm
☐ CDN
☐ Config

If you're already using [npm](#) and a module bundler such as [webpack](#) or [Rollup](#), you can run the following command to install the latest SDK ([Learn more](#)):

```
$ npm install firebase
```



Then, initialize Firebase and begin using the SDKs for the products you'd like to use.

```
// Import the functions you need from the SDKs you need
import { initializeApp } from "firebase/app";
// TODO: Add SDKs for Firebase products that you want to use
// https://firebase.google.com/docs/web/setup#available-libraries

// Your web app's Firebase configuration
const firebaseConfig = {
  apiKey: "AIzaSyBrfB_-Zku0UBMNMqldUedcmKpg0Wn1o84",
  authDomain: "restfulapi-demo01.firebaseio.com",
  projectId: "restfulapi-demo01",
  storageBucket: "restfulapi-demo01.appspot.com",
  messagingSenderId: "249666661465",
  appId: "1:249666661465:web:5add8ea929d17777b45548"
};

// Initialize Firebase
const app = initializeApp(firebaseConfig);
```

▼ Bai12	●
> assets	●
> components	●
▼ config	●
JS firebase.js	U
> node_modules	
🔗 .gitignore	A
JS App.js	A
{} app.json	M
🔗 babel.config.js	A
{} package-lock.json	M
{} package.json	M

```

import { initializeApp } from "firebase/app";
import { getFirestore, query, orderBy, limit } from
"firebase/firestore";

const firebaseConfig = {
  apiKey: "AIzaSyBrfB_-Zku0UBMNMqldUedcmKPg0Wn1o84",
  authDomain: "restfulapi-demo01.firebaseio.com",
  projectId: "restfulapi-demo01",
  storageBucket: "restfulapi-demo01.appspot.com",
  messagingSenderId: "249666661465",
  appId: "1:249666661465:web:5add8ea929d17777b45548"
};

// Initialize Firebase
const app = initializeApp(firebaseConfig);
const db = getFirestore(app);

export default {db};

```

```
const openGallery = async () =>
{

    let pickerResult = await ImagePicker.launchImageLibraryAsync({
        mediaTypes: ImagePicker.MediaTypeOptions.Images,
        allowsEditing: true,
        aspect: [1, 1],
        quality: 1
    });
    if (!pickerResult.canceled)
    {
        console.log(pickerResult.assets[0].uri);
        setSelectedImage({ localUri: pickerResult.assets[0].uri });

        let blobFile =await  getBlobFromUri(pickerResult.assets[0].uri);
        console.log('blob',blobFile)
        await uploadBlobFile(blobFile);
    }
};
```

```
const getBlobFromUri = async (uri) => {  
  const blob = await new Promise( (resolve, reject) => {  
    const xhr = new XMLHttpRequest();  
    xhr.onload = function () {  
      resolve(xhr.response);  
    };  
    xhr.onerror = function (e) {  
      reject(new TypeError("Network request failed"));  
    };  
  
    xhr.responseType = "blob";  
    xhr.open("GET", uri, true);  
    xhr.send(null);  
  });  
  
  return blob;  
};
```

```

const uploadBlobFile = async (blob) => {
  const imgName = "img-w-" + new Date().getTime();
  const storage = getStorage();
  const metadata = {
    contentType: 'image/jpeg',
  };
  const storageRef = ref(storage, `images/${imgName}.jpg`);
  console.log("uploading file", imgName);
  const uploadTask = uploadBytesResumable (storageRef, blob, 'base64');
  uploadTask.on('state_changed',
    (snapshot) => {
      const progress = (snapshot.bytesTransferred / snapshot.totalBytes) * 100;
      console.log('Upload is ' + progress + '% done'); },
    (error ) => {console.log(error) },
    () => {
      // Upload completed successfully, now we can get the download URL
      getDownloadURL(uploadTask.snapshot.ref).then((downloadURL) => {
        console.log('File available at', downloadURL);
        //setSelectedImage(downloadURL);
      });
    }
  );
}

```